



Universidade do Minho

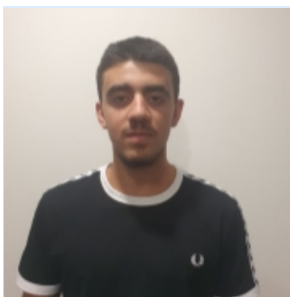
Sistemas Operativos

Trabalho Prático

Licenciatura em Engenharia Informática - 2022/2023

Grupo 99

João Barroso
A95195



Afonso Bessa
A95225



Francisco Claudino
A89493



1 Introdução

No âmbito da Unidade Curricular de Sistemas Operativos, realizamos este projeto que consiste na implementação de um serviço de monitorização de programas executados numa máquina, através de um sistema cliente/servidor (monitor/tracer) no qual são implementadas diversas funcionalidades.

Este sistema deve permitir aos utilizadores a execução de programas, através do cliente, e obter o seu tempo de execução (o tempo total desde que o utilizador pede ao cliente para correr o programa até ao término do mesmo).

Da mesma forma, este sistema deve permitir que um administrador de sistemas consiga consultar, através do servidor, todos os programas que se encontram atualmente em execução, incluindo o tempo despendido pelos mesmos.

Finalmente, o servidor deve também permitir a consulta de estatísticas sobre programas já terminados (por exemplo, o tempo de execução agregado de um certo conjunto de programas).

Consequentemente, estas funcionalidades foram testadas através de várias instruções de teste.

2 Estruturas Implementadas

Durante a realização do trabalho foram necessárias implementar várias estruturas que permitissem:

1. Enviar informação do início e do fim da execução de programas (ou pipeline de programas), bem como do seu output, timestamps de início e fim de execução, nome do pipe com nome para onde enviar a resposta e número de PID's, caso se trate de uma estatística "stats".
2. Guardar informação sobre os vários programas que estão a ser executados.

Desta forma e para responder as necessidades do sistema cliente/servidor, foram implementadas as seguintes estruturas, respetivamente:

1. Estrutura Programa que envia a informação sobre os programas executados ou os pedidos de estatísticas e que tem como campos os seguintes :
 - (a) int pid - corresponde ao pid do processo;
 - (b) int numeroPids - corresponde aos PID's pedidos caso seja uma estatística "stats";
 - (c) char fd[32] - o nome do pipe com nome caso seja uma estatística;
 - (d) int flag - flag que indica o tipo de pedido;
 - (e) char programa[64] - a informação sobre o pedido;
 - (f) long tempo - o timestamp em long int;
 - (g) double seconds - o timestamp em double segundos;
 - (h) double microseconds - o timestamp em double milissegundos;
2. Estrutura ProgramaLista que corresponde a uma lista ligada que armazena os programas que estão atualmente em execução e tem como campos os seguintes :
 - (a) struct Programa *p - o programa atual;
 - (b) struct ProgramaLista *prox - corresponde a um apontador para o próximo programa;

Estas estruturas encontram-se definidas no ficheiro "Estruturas.h", bem como os header files das funções auxiliares que utilizamos para efetuar parsing de informação bem como para ajudar no funcionamento do monitor e do tracer (serão abordadas nas próprias secções). A implementação destas funções encontram-se no ficheiro "Estruturas.c".

3 Ficheiros Utilizados

Neste Projeto, para além dos ficheiros auxiliares abordados no capítulo anterior, utilizamos também a pasta "PIDS-folder" para guardar um histórico de todos os programas executados. Nesta pasta guardamos um ficheiro por programa executado, cujo nome corresponde ao PID do processo que executou o programa e o conteúdo do ficheiro consiste no nome do programa (ou da pipeline de programas) bem como no tempo de execução do mesmo.

Para além desta pasta de ficheiros, utilizamos também um pipe com nome (FIFO). Este estabelece uma comunicação unidirecional entre o cliente e o servidor, onde o tracer lê o pedido do cliente e escreve as informações sobre o início e o fim da execução desse pedido no pipe e o servidor lê e armazena essa informação.

Também, durante a execução, possuímos vários ficheiros temporários, nomeadamente pipes com nomes que são usados para passar a informação das estatísticas quando são pedidas pelo administrador de sistemas. Esses pipes têm o nome "FIFO" mais o próprio nome da estatística a ele associada e o numero do PID do processo que o criou e requisitou, para controlo de concorrência (que será abordado no devido capítulo). De referir que estes pipes são também eles unidirecionais e onde o servidor escreve o resultado a estatística pedida e o tracer lê e imprime no Standard Output do administrador de sistemas.

Por fim, também possuímos o ficheiro "PID.txt" que serve para armazenar o PID atual. Este ficheiro serve para o tratamento do controlo de PID's repetidos e será abordado com mais detalhe no respetivo capítulo.

4 Funcionamento do Monitor

O monitor consiste no servidor do sistema que recebe informação sobre os programas executados por cada processo, escrevendo para um ficheiro a informação sobre o PID do processo que o executou, o tempo que demorou durante a execução e o nome do programa (ou pipeline de programas) e também recebe pedidos de estatísticas por parte dos administradores de sistemas.

Para ser capaz de efetuar operações tão distintas, o monitor está dividido através de um switch - case com 5 casos que passo a citar:

1. Case 0 ocorre quando o monitor recebe informação sobre um programa ou uma pipeline de programas que começaram a ser executadas, adicionando a informação do programa á lista ligada ou acabaram a sua execução.
2. Case 1 ocorre quando o monitor recebe informação sobre um programa ou uma pipeline de programas que removendo a informação da lista ligada e escrevendo um ficheiro na pasta PIDS-folder com o nome do programa / pipeline de programas e com o tempo de execução.
3. Case 2 ocorre quando recebe um pedido de status ao qual ele chama as funções que efetuam esta estatística e escreve no pipe com nome a resposta da estatística.
4. Case 3 ocorre quando recebe um pedido de stats-time ao qual ele chama as funções que efetuam esta estatística e escreve no pipe com nome a resposta da estatística.
5. Case 4 ocorre quando recebe um pedido de stats-command ao qual ele chama as funções que efetuam esta estatística e escreve no pipe com nome a resposta da estatística.
6. Case 5 ocorre quando recebe um pedido de stats-uniq ao qual ele chama as funções que efetuam esta estatística e escreve no pipe com nome a resposta da estatística.

5 Funcionamento do Tracer

O tracer oferece uma interface com o utilizador via linha de comandos. É através do tracer que o utilizador pode pedir a execução de um programa único ou de uma pipeline de programas. É também através do tracer que um administrador de sistemas consiga consultar a informação de todos os programas que se encontram em execução, bem como o PID do processo responsável pela sua execução e o tempo despendido na execução.

Consequentemente, também é possível que o administrador consiga consultar outras estatísticas relacionadas com programas já executados, sendo estas :

1. O tempo de execução agregado de um certo número de programas.
2. O número de vezes que um certo programas foi executado.
3. O nome de todos os programas únicos que foram executados.

Estas estatísticas requerem que o administrador indique os PID's aos quais pretende obter estas estatísticas.

O funcionamento do tracer baseia-se no parsing da informação que recebe como argumentos na linha de comandos. Esta análise passa por avaliar qual o segundo argumento (`argv[1]`) enviado pelo utilizador (através do uso da função `strcmp`). A partir deste ponto, se o argumento for "execute" terá de avaliar o terceiro argumento enviado (`argv[2]`) pelo utilizador (usando novamente a função `strcmp`).

Após ser avaliado o pedido que é feito pelo utilizador, procede-se ao parsing e tratamento dos argumentos restantes e da execução do pedido, quer seja a execução de um único programa ou de uma pipeline de programas quer seja uma estatística. Abordaremos com mais detalhe a execução de cada um dos diferentes pedidos nos seus respetivos capítulos.

5.1 Executar Programa Único (`execute -u`)

Sempre que o utilizador requisita a execução de um programa único, é chamada uma função que faz o parsing dos argumentos e que constrói um array de strings onde o primeiro elemento consiste no nome do programa e os elementos seguintes consistem nos argumentos (se existirem) do programa.

Para a execução do programa é utilizada a primitiva `execvp()` onde se passa o primeiro elemento do array como primeiro argumento e o array como segundo argumento. É efetuado um `fork()` antes da execução do programa, onde o processo filho executa o programa e o processo pai envia informação sobre o início e o fim da execução.

No início da execução é imprimido no Standard Output o número do PID do processo que executa o programa e é enviada a informação ao servidor através do "FIFO". Mal a execução termina, envia-se a informação ao servidor que o programa foi executado através do "FIFO" (tal como no início da execução) e é impresso no Standard Output do utilizador o output do programa bem como o tempo de execução do mesmo.

5.2 Executar Pipeline de Programas (execute -p)

Sempre que o utilizador requisita a execução de uma pipeline de programas, da mesma forma que é efetuada na execução única de um programa, é chamada uma função que efetua o parsing do argumento onde contém a informação sobre a pipeline de programas e dos seus argumentos. Esta função constrói um array de strings onde cada string corresponde a um programa da pipeline e aos seus respetivos argumentos.

Da mesma forma, é chamada uma função que conta quantos programas constituem a pipeline e é executado um ciclo for com esse número de iterações. Em cada iteração é criado um pipe anónimo que serve para comunicar entre o filho e o pai (excetuando na execução do último programa da pipeline, pois já não é necessário, visto que o pipe do penúltimo processo já está criado e apenas necessita de ler o output do penúltimo programa e após a execução do ultimo programa imprimir no Standard Output), sendo que podemos ter 3 casos:

- O caso em que o programa é o primeiro da pipeline, ou seja, apenas é necessário que o processo filho feche o descritor de leitura do pipe anónimo (pois este apenas vai escrever o output do programa no pipe), redirecionar o Standard Output para o descritor de escrita do pipe anónimo e de seguida fechar o descritor de escrita do pipe anónimo pois já efetuou o redirecionamento, não sendo mais necessário por parte do filho escrever no pipe. Desta forma, mal o processo filho execute o programa o output deste é escrito no pipe anónimo e o filho dá exit. Por sua vez, o processo pai necessita apenas de esperar pelo filho e de fechar o descritor de escrita do pipe pois apenas vai ler o output do programa executado no filho.
- O caso em que o programa é o ultimo programa da pipeline, ou seja, apenas é necessário que o processo filho redirecione a leitura do pipe para o seu Standard Input (visto que necessita do output do penúltimo programa como argumento do último programa) e de seguida procede ao fecho de ambos os descritores do pipe. O processo pai, para além de esperar pelo processo filho, tem também de fechar ambos os descritores do pipe, pois o output do último programa e, por conseguinte, da pipeline de programas será imprimido no Standard Output.
- O caso em que o programa não é o primeiro nem o último da pipeline, o que implica que o processo filho feche o descritor de leitura do pipe que vai usar para comunicar com o seu pai, redireciona a leitura do pipe para o seu Standard Input (visto que necessita do output do programa anterior como argumento do programa "atual"), fecha o descritor de leitura deste pipe, redireciona o seu Standard Output para o descritor de escrita do pipe e de seguida fecha este mesmo descritor. Desta forma, mal o processo filho execute o programa o output deste é escrito no pipe anónimo e o filho dá exit. Quanto ao processo pai, para além de esperar pelo filho tem ainda de fechar o descritor de leitura do pipe anterior e fechar o descritor de escrita do pipe "atual", visto que apenas necessita de ler deste pipe.

Após a conclusão deste ciclo for, ocorre outro ciclo for com o mesmo número de iterações mas que serve apenas para esperar por todos os processos filho.

Após a conclusão de ambos os ciclos for é imprimido no ecrã o output da pipeline (esta impressão ocorre logo após a primitiva `execvp()` do último programa) bem como o seu tempo de execução.

É também de referir que, em cada caso, é chamada a mesma função chamada na execução de programas únicos, para obter um array de strings onde o primeiro elemento é o nome do programa e os restantes são os seus argumentos, por forma a passar este o primeiro elemento deste array bem como o array completo a primitiva "execvp" para execução de cada programa.

5.3 Status (status)

Quando um administrador de sistemas requisita a estatística do "status", é criado no tracer um pipe com nome cujo nome consiste em "FIFO"+ "status"+ o PID do processo. O nome deste pipe é escrito no FIFO tal como a informação de que é pretendido efetuar um "status".

Mal o servidor lê esta informação, percorre a lista ligada onde estão armazenados os programas em execução e calcula a diferença de tempo entre o timestamp que recebeu com a informação do status e o timestamp de início de cada programa. Cada vez que lê a informação relativa a um programa ele cria uma string com a informação do PID, o nome do programa e o tempo de execução atual e concatena com a string dos outros programas (excetuando o caso em que apenas existe um programa a executar ou que é o primeiro que percorreu, sendo que nestes casos ele cria apenas a string, não havendo necessidade de concatenação). A concatenação é efetuada através da função `strcat()`.

Após percorrer a lista ligada e de ter a string construída, este abre o pipe com nome "passado" pelo tracer e envia a string com o resultado do "status". O tracer mal lê a informação que o servidor escreveu, elimina o pipe com nome através da primitiva `unlink()` e imprime o a string do resultado do status no Standard Output do administrador de sistemas.

5.4 Stats Time (stats-time)

Sempre que um administrador de sistemas requisita a estatística "stats-time", é criado no tracer um pipe com nome cujo nome consiste em "FIFO"+ "stats-time"+ o PID do processo. O nome deste pipe é escrito no FIFO tal como a informação de que é pretendido efetuar um "stats-time" bem como a informação sobre quais os PID's que se pretende obter o tempo agregado de execução.

Mal o servidor lê esta informação, chama uma função para contar o número de PID's e efetua um ciclo for com o número de iterações igual ao número de PID's onde em cada iteração abre o ficheiro correspondente ao nome do PID, com a primitiva `open()` e utiliza a primitiva `lseek()` para obter a informação relativa ao tempo de execução do programa, procedendo de seguida ao fecho do ficheiro através da primitiva `close()`. Esta informação é adicionada ao contador e a string que contém o número dos PID's é atualizada com o `strtok()`.

Após o ciclo é construída a string com o resultado da estatística "stats-time", é aberto o pipe com nome "passado" pelo tracer e é escrita a string correspondente ao resultado da estatística. O tracer mal lê a informação escrita pelo servidor, elimina o pipe com nome através da primitiva `unlink()` e imprime a string do resultado do "stats-time" no Standard Output do administrador de sistemas.

5.5 Stats Command (stats-command)

Sempre que um administrador de sistemas requisita a estatística "stats-command", é criado no tracer um pipe com nome cujo nome consiste em "FIFO"+ "stats-command"+ o PID do processo. O nome deste pipe é escrito no FIFO tal como a informação de que é pretendido efetuar um "stats-command" bem como a informação sobre quais os PID's que se pretende obter o tempo agregado de execução.

Mal o servidor lê esta informação, chama uma função para contar o número de PID's e efetua um ciclo for com o número de iterações igual ao número de PID's onde em cada iteração abre o ficheiro correspondente ao nome do PID, com a primitiva "open()" e utiliza a primitiva "lseek()" para obter a informação relativa ao nome do programa, procedendo de seguida ao fecho do ficheiro através da primitiva "close()". Esta informação é adicionada ao contador (se o strcmp() entre o nome do programa e o nome do programa pedido na estatística for "true") e a string que contém o número dos PID's é atualizada com o strtok().

Após o ciclo é construída a string com o resultado da estatística "stats-command", é aberto o pipe com nome "passado" pelo tracer e é escrita a string correspondente ao resultado da estatística. O tracer mal lê a informação escrita pelo servidor, elimina o pipe com nome através da primitiva "unlink()" e imprime a string do resultado do "stats-command" no Standard Output do administrador de sistemas.

5.6 Stats Unique (stats-uniq)

Sempre que um administrador de sistemas requisita a estatística "stats-uniq", é criado no tracer um pipe com nome cujo nome consiste em "FIFO"+ "stats-uniq"+ o PID do processo. O nome deste pipe é escrito no FIFO tal como a informação de que é pretendido efetuar um "stats-uniq" bem como a informação sobre quais os PID's que se pretende obter o tempo agregado de execução.

Mal o servidor lê esta informação, chama uma função para contar o número de PID's e efetua um ciclo for com o número de iterações igual ao número de PID's onde em cada iteração abre o ficheiro correspondente ao nome do PID, com a primitiva "open()" e utiliza a primitiva "lseek()" para obter a informação relativa ao nome do programa, procedendo de seguida ao fecho do ficheiro através da primitiva "close()". Esta informação é adicionada ao array de strings (se o strcmp() entre o nome do programa e os nomes dos programas contidos no array de strings for "false") e a string que contém o número dos PID's é atualizada com o strtok().

Após o ciclo é construída a string com o resultado da estatística "stats-uniq" através de concatenações das strings do array de strings, é aberto o pipe com nome "passado" pelo tracer e é escrita a string correspondente ao resultado da estatística. O tracer mal lê a informação escrita pelo servidor, elimina o pipe com nome através da primitiva "unlink()" e imprime a string do resultado do "stats-uniq" no Standard Output do administrador de sistemas.

6 Controlo de PID's

Durante os testes que foram efetuados as diversas funcionalidades deste sistema cliente/servidor deparamo-nos com um eventual problema que poderia surgir. Este problema consiste na repetição de PID's durante a execução de vários pedidos, o que levaria a um problema de coerência aquando da escrita dos ficheiros na pasta PIDS-folder.

A solução encontrada para este problema foi a de se atribuir os PID's de forma "manual", sem recorrer aos PID's que o Sistema Operativo criava. Esta variável é mantida no ficheiro "PID" na pasta "bin" e sempre que é utilizada para um pedido é incrementada e reescrita no ficheiro. Desta forma não existe a possibilidade de repetição de PID's nos processos.

7 Controlo de Concorrência

Ao executar as tarefas de forma concorrente, foi possível notar que os outputs das estatísticas estavam a ser escritos também de forma concorrente e, devido a este facto, os outputs pedidos pelos utilizadores poderiam não ser coerentes. Assim, foi preciso utilizar um mecanismo de controlo de concorrência para evitar esta situação.

A estratégia utilizada foi a de ser criado um pipe com nome por cada administrador de sistemas que requisitava uma estatística com o nome "FIFO"+ nome da estatística pedida + PID do processo. O nome do pipe com nome seria passado na estrutura com a informação da estatística pretendida (através do FIFO) e a resposta seria enviada por este pipe, sendo que o tracer que enviou o pedido ficaria a ler deste pipe, destruindo-o mal fosse lido o resultado da estatística pedida. Desta forma, não há possibilidade de outputs errados / não coerentes serem enviados a administradores de sistema.

8 Conclusão

De facto, mesmo tendo realizado todas as tarefas presentes no enunciado (apesar de existir erros no status e no stats-command) há alguns pontos que gostaríamos de apontar tanto de forma positiva como de forma negativa.

Primeiramente, gostaríamos de destacar a forma como tratamos as tarefas que são tratadas no servidor (as estatísticas), sendo que, tornamos possível preservar o estado do mesmo, após ser fechado. Por conseguinte, destacaríamos também a implementação das tarefas adicionais através da leitura e escrita de e para ficheiros da informação relevante dos pedidos efetuados previamente.

Por outro lado, como ponto menos positivo, não poderíamos deixar de indicar a forma como implementamos o controlo de concorrência sobre os pedidos de estatísticas ao servidor. A solução arranjada funciona mas consome bastante memória, pois por cada pedido que é efetuado é criado um pipe com nome. No entanto, foi a opção que consideramos ser a melhor para resolver este problema.

Em suma, Após finalizarmos todas as tarefas apercebemos-nos de dois erros que não conseguimos resolver até a data de entrega. O primeiro no comando stats-command, caso o programa que passemos seja uma pipeline não irá funcionar, isto porque nos esquecemos que poderia ser uma pipeline e portanto tratamos da string só para programas únicos. O segundo é na nossa lista ligada que da origem ao status, o primeiro elemento dessa lista é permanente e não o conseguimos apagar da mesma. Isto faz com que o status imprima sempre esse programa mesmo após ele já ter finalizado. Testamos muitas formas diferentes de resolver o problema mas foi algo que descobrimos à última da hora e não ficou corrigido. Apesar deste dois erros, as funcionalidades estão todas funcionais e estamos muito contentes com o resultado final pois exigiu imenso trabalho da nossa parte. Sentimentos que num futuro trabalho que possa envolver system calls conseguiríamos obter ainda melhor desempenho que neste trabalho, pois apesar de o tempo disponibilizado ser justo para

a realização do mesmo, demoramos algum tempo a dominar as system calls e como usá-las e acabamos por não ter conseguido ter tudo sem erros. Na nossa visão, em geral, foi um trabalho bem conseguido, sendo que, aplicamos conhecimentos como pipes anónimos, pipes com nome, fork, exec, redireccionamento e ainda conseguimos resolver o problema da concorrência sobre os pedidos de estatísticas efetuadas ao servidor, ainda que, possa não ser a melhor implementação.